

Guia Completo — "Pequenos Smarters"

(versão atualizada)

Para: Estefanie e Rafaela **O que este guia é:** o passo a passo pra terminar o jogo, batendo com o projeto **como ele está hoje** (depois da reforma visual). Traz a divisão do trabalho **por ilha** e os passo a passos de imagem, som, dublagem, animação e história.

Personagens: Ziggy (leão), Pipo (macaco), Lina (elefanta), Doutor Preguiça (antagonista). **Ordem das ilhas:** Natureza → Deserto → Gelo → Ventos → Fogo → Lagos. **Inspiração de conceito:** levemente inspirado no **Dragon Mania** (ilhas flutuantes / mapa por mundos). É só a fagulha da ideia.

SUMÁRIO

- Parte 0 — Estado atual do projeto
 - Parte 1 — Tecnologias e ferramentas
 - Parte 2 — Git em dupla (essencial)
 - Parte 3 — Divisão do trabalho (por ilha) e o "contrato"
 - Parte 4 — Como o app está montado hoje (arquitetura real)
 - Parte 5 — O motor de jogo (6 componentes) e como criar uma ilha nova
 - Parte 6 — Imagens: do Figma para o app (passo a passo)
 - Parte 7 — Sonorização (efeitos + música) — passo a passo
 - Parte 8 — Dublagem / narração (voz) — passo a passo
 - Parte 9 — Animação e vida na tela (borboletas, árvores, personagens) — passo a passo
 - Parte 10 — Roteiro / história interativa — passo a passo
 - Parte 11 — Cronograma sugerido
 - Apêndice — Cola de comandos do Git
-

PARTE 0 — Estado atual do projeto

✓ Pronto e funcionando

- App **horizontal** (landscape), rodando no **Expo Go (SDK 54)**.
- **Supabase** ligado: cadastro, login, sessão que lembra, salvar progresso, config.
- **Fluxo de telas** completo: Home → Login/Cadastro → Boas-vindas (Ziggy) → Ilha → Fase → Resultado.
- **Sistema visual** com o design do Figma: fundo de céu, logo, personagens, fontes (Concert One + Baloo 2), barra do topo padrão, popup de configurações com **Sair**.
- **Ilha da Natureza 100% pronta** (é o **modelo** pra copiar): ilha desenhada como cenário principal + 3 fases jogáveis + tela de Resultado com estrelas.
- **Motor de jogo** (6 componentes) — os que a Natureza usa (`SelecaoUnica` , `EncontrarAlvos`) já estão no visual novo.
- **Pipeline de imagens**: PNG em alta (otimizado) e SVG já configurados.

➡ SOON O que falta

- Construir as **outras 5 ilhas** (Deserto, Gelo, Ventos, Fogo, Lagos) — divididas entre vocês (Parte 3).
 - **Artes das fases de imagem** feitas no Figma e exportadas (Parte 6).
 - **Sonorização** (Parte 7), **dublagem** (Parte 8), **animação** (Parte 9), **história** (Parte 10).
-

PARTE 1 — Tecnologias e ferramentas

Categoria	Ferramenta	Observação
App (framework)	React Native + Expo (SDK 54)	app mobile, roda no celular via Expo Go
Linguagem	JavaScript	
Navegação	React Navigation	pilha de telas
Backend / Banco	Supabase	Auth + banco PostgreSQL + segurança RLS
Design	Figma	telas, personagens, ilhas
Vetores/Imagens	react-native-svg + PNG em alta	
Versionamento	Git + GitHub	branches, PRs
Gestão	Trello	
Editor	VS Code	

⚠ **Não** liste "Node.js" como tecnologia do app: vocês removeram o backend em Node/Express e trocaram pelo Supabase. O Node só existe nos bastidores das ferramentas (npm/Expo), não é o backend do jogo.

PARTE 2 — Git em dupla (essencial)

Conceitos: `main` = versão oficial que sempre funciona; `branch` = sua cópia de trabalho; `Pull Request (PR)` = pedir pra juntar sua branch na main.

📌 `.gitignore` (na raiz) precisa conter no mínimo:

```
node_modules/  
.expo/  
dist/  
web-build/
```

```
*.log
.DS_Store
```

Fluxo do dia a dia:

```
git checkout main && git pull origin main      # 1. pega o mais novo
git checkout -b feat/ilha-deserto              # 2. cria sua branch
# 3. ... programa e testa ...
git add . && git commit -m "mensagem clara"    # 4. salva (várias vezes)
git push origin feat/ilha-deserto              # 5. sobe
# 6. no GitHub: abre o PR da sua branch -> main, a dupla revisa e faz Merge
```

Combinados (colem no Trello): nunca commitar na `main`; `git pull` antes de começar; commits pequenos e frequentes; só abrir PR quando o app roda; avisar a dupla antes de mexer em arquivo compartilhado (`App.js` , `AppNavigator.js` , os componentes do motor).

PARTE 3 — Divisão do trabalho (por ilha)

A **plataforma** (login, Supabase, navegação, painel, configurações) e o **motor de jogo** (os 6 componentes) **já estão prontos**. O que sobra agora é construir as **ilhas** — então a divisão mais natural é **por ilha**: cada uma fica dona de ilhas inteiras.

A **Ilha da Natureza** já está pronta e serve de **modelo** pra copiar. Faltam 5:

Pessoa	Ilhas	Observação
Rafaela	Deserto, Gelo, Ventos	Deserto e Gelo são de letras/sílabas (sem arte nova); Ventos usa imagens
Estefanie	Fogo, Lagos	as duas usam imagens (precisam de arte do Figma)

É só uma **sugestão** — dividam como preferirem. O importante é cada uma pegar **ilhas inteiras**.

Por que dividir por ilha é seguro no Git

Cada ilha mora em **arquivos próprios**, então vocês quase nunca editam o mesmo arquivo: -

```
src/data/<ilha>.js - src/game/fases/<Ilha>Fase1.js , Fase2.js , Fase3.js -
```

`src/screens/islands/<Ilha>Screen.js` - `assets/images/ilha<Ilha>.png`

Arquivos compartilhados (atenção): - `src/navigation/AppNavigator.js` — **já registra todas as ilhas e fases.** Não precisa mexer. - Os **6 componentes do motor** (`src/game/*.js`) são compartilhados. `SelecaoUnica` e `EncontrarAlvos` já estão no visual novo; os outros 4 ainda estão no antigo. **Quem precisar de um deles primeiro, deixa ele bonito** (usando o `SelecaoUnica` como modelo), commita e **avisa a dupla** pra não refazerem em dobro.

Qual componente cada ilha usa (pra saber o que pode precisar arrumar): | Ilha | Fase 1 | Fase 2 | Fase 3 | |---|---|---|---| | Natureza  | EncontrarAlvos | SelecaoUnica | SelecaoUnica | | Deserto | EncontrarAlvos | SelecaoUnica | OrdenarSequencia  | | Gelo | JogoMemoria  | SelecaoUnica | OrdenarSequencia  | | Ventos | SelecaoUnica (img) | SelecaoUnica | OrdenarSequencia  | | Fogo | SelecaoUnica (img) | SelecaoUnica | OrdenarSequencia  | | Lagos | SelecaoUnica (img) | LigarColunas  | DigitarPalavra  |

 = componente ainda no visual antigo (a primeira pessoa que precisar dele deixa bonito e avisa). Repare que **OrdenarSequencia** é usado por 4 ilhas — quem chegar nele primeiro resolve pra todas.

O "contrato" (continua valendo)

Toda fase, ao terminar, chama a tela de Resultado com este pacote:

```
navigation.replace('Result', {
  estrelas,                // 1, 2 ou 3
  erros,                  // número de erros
  ilha: 'deserto',        // identifica a ilha
  fase: 'fase2',          // identifica a fase
  faseAtual: 'DesertoFase2', // rota desta fase (pra "Jogar novamente")
  proximaFase: 'DesertoFase3', // próxima fase; na última fase da ilha, a rota da PRÓXIMA ilha
  mensagem: 'Muito bem!',
});
```

As rotas seguem o padrão **NomellhaFaseNúmero**. O fluxo é **linear**: a última fase de uma ilha aponta pra próxima ilha (ex.: a última do Deserto → `'Gelo'`).

PARTE 4 — Como o app está montado hoje (arquitetura real)

Navegação

É uma pilha só que começa **SEMPRE** na Home (`src/navigation/AppNavigator.js`): - **Home** é a primeira tela sempre. - O botão ► da Home: se **já tem login**, vai pro **Welcome (Ziggy)**; se não, vai pro **Login**. - Login/Cadastro, ao dar certo, fazem `navigation.replace('Welcome')`. - **Sair** (popup de config) faz logout e volta pra Home. - O `AuthContext` (`src/navigation/AuthContext.js`) guarda a sessão.

Sistema visual

- `src/utils/cores.js` — paleta central.
- `src/utils/tema.js` — as fontes: `titulo` (Baloo 2), `subtitulo`, `texto` (Concert One).
- `src/components/Fundo.js` — fundo de tela (céu por padrão).
- `src/components/BarraTopo.js` — barra do topo: ⚙ config + logo à esquerda; estrelas + "Painel dos Pais" + 🏠 à direita. Respeita a **área segura** do celular; o 🏠 leva pra ilha atual (prop `home`).
- `src/components/ConfigPopup.js` — popup de Som/Voz/Créditos/**Sair**.
- Tudo é **horizontal** (`app.json` → `"orientation": "landscape"`).

Onde ficam as coisas

```
assets/images/    ← fundos, ilhas, personagens, logo, botões, palavras/, frases/
assets/sounds/    ← (vazio por enquanto) efeitos e vozes vão aqui
src/components/   ← Fundo, BarraTopo, ConfigPopup, StarRating
src/screens/      ← Home, Login, Cadastro, Welcome, Result, Settings, Parents
src/screens/islands/ ← as 6 telas de ilha
src/game/         ← os 6 componentes do motor
src/game/fases/   ← as 18 fases
src/data/         ← o conteúdo das fases
src/services/     ← supabase, auth, progresso, config (e futuramente som, voz)
src/utils/        ← cores, estrelas, tema
```

PARTE 5 — O motor de jogo e como criar uma ilha nova

Os **6 componentes** (em `src/game/`) são o coração:

Componente	Serve para
<code>SelecaoUnica</code>	clicar na opção certa (letra, sílaba, palavra ou imagem)
<code>EncontrarAlvos</code>	tocar em vários itens que seguem uma regra (ex: as vogais)
<code>JogoMemoria</code>	virar cartões e achar pares
<code>OrdenarSequencia</code>	tocar itens na ordem certa
<code>DigitarPalavra</code>	digitar e validar letra por letra
<code>LigarColunas</code>	ligar cada frase à imagem certa

Receita para fazer uma ilha (ex.: Deserto)

1. **Dados** — preencha `src/data/deserto.js` (já existe um esqueleto).
2. **3 fases** — em `src/game/fases/` já existem `DesertoFase1/2/3.js`. Confira `ilha`, `fase`, `faseAtual` e `proximaFase` (a última fase aponta pra próxima ilha).
3. **Tela da ilha** — reescreva `src/screens/islands/DesertoScreen.js` no molde da `NaturezaScreen.js`: `Fundo` (céu) + `BarraTopo` + a **arte da ilha** no centro + os 3 botões de fase posicionados sobre ela.
4. **Arte da ilha** — exporte do Figma (Parte 6) e salve como `assets/images/ilhaDeserto.png`.
5. **Registrar** — o `AppNavigator.js` já registra tudo; só confira.
6. **Testar** e commitar numa branch `feat/ilha-deserto`.

Opções com imagem (fases de figura)

O `SelecaoUnica` aceita dois jeitos:

```
// PNG (arte detalhada do Figma):
{ id: 'gato', imagem: require('../../assets/images/palavras/gato.png') }

// OU SVG (desenho simples/plano):
import Gato from '../../assets/images/palavras/gato.svg';
{ id: 'gato', Imagem: Gato }
```

PARTE 6 — Imagens: do Figma para o app (passo a passo)

1. **No Figma**, selecione o elemento/quadro.

2. Pannel **Export** → +.
3. Tamanho **certo pra celular** (evite arquivos gigantes): - **Fundos/cenários** → PNG, ~**1600px** de largura. - **Ilhas** → PNG, ~**1400px**. - **Personagens** → PNG, ~**600px**. - **Objetos pequenos** (letras, ícones, borboleta) → PNG ~**300–500px**, ou **SVG** se for plano.
4. Salve com **nomes sem espaço/acentos** em `assets/images/` (ou subpastas `palavras/`, `frases/`).
5. No código, `require('../assets/images/...png')`.

⚠ Regra de ouro do tamanho

Imagem de **11 MB trava o celular**. Um fundo precisa de ~1600px, não 4000px. Se um PNG passar de ~2 MB, reduza (reexporte menor no Figma ou comprima em *squoosh.app*).

Dica para animar (Parte 9)

Pra **borboleta voando** e **árvore balançando**, exporte esses elementos como **PNG separados** (fundo transparente). Só dá pra animar o que é uma imagem própria — não um pedaço de dentro da imagem "chapada" da ilha.

PARTE 7 — Sonorização (efeitos + música) — passo a passo

Objetivo: sons de **acerto, erro, toque, fanfarra** no Parabéns e (opcional) **música de fundo**.

Passo 1 — Conseguir os sons (legalmente)

Use áudios **livres de direitos** (CC0 / royalty-free): **mixkit.co/free-sound-effects**, **pixabay.com/sound-effects**, **freesound.org** (confira a licença). Sons **curtos** (~1s). Guarde a lista pra pôr nos **Créditos**.

Passo 2 — Organizar os arquivos

Salve em `assets/sounds/` (em .mp3):

```
assets/sounds/acerto.mp3
assets/sounds/erro.mp3
assets/sounds/toque.mp3
assets/sounds/parabens.mp3
assets/sounds/musica.mp3  (opcional, loop)
```

⚠️ Crie o `som.js` **só depois** de pôr os arquivos (o `require` de arquivo inexistente quebra o app).

Passo 3 — Serviço de som (`src/services/som.js`)

```
// src/services/som.js
import { createAudioPlayer } from 'expo-audio';

let somLigado = true;
export function setSomLigado(v) { somLigado = v; }

const players = {
  acerto: createAudioPlayer(require('../assets/sounds/acerto.mp3')),
  erro: createAudioPlayer(require('../assets/sounds/erro.mp3')),
  toque: createAudioPlayer(require('../assets/sounds/toque.mp3')),
  parabens: createAudioPlayer(require('../assets/sounds/parabens.mp3')),
};

export function tocar(nome) {
  if (!somLigado) return;
  const p = players[nome];
  if (!p) return;
  try { p.seekTo(0); p.play(); } catch (e) {}
}

const musica = createAudioPlayer(require('../assets/sounds/musica.mp3'));
musica.loop = true;
export function tocarMusica() { if (somLigado) { try { musica.play(); } catch (e) {} } }
export function pararMusica() { try { musica.pause(); } catch (e) {} }
```

✦ O `expo-audio` (SDK 54) mudou em relação ao antigo `expo-av`. Se algum método tiver outro nome, confira: <https://docs.expo.dev/versions/v54.0.0/sdk/audio/>

Passo 4 — Tocar nos momentos certos

- Em `SelecaoUnica.js` / `EncontrarAlvos.js`: `tocar('acerto')` no acerto, `tocar('erro')` no erro.
- Em `ResultScreen.js` (no `useEffect`): `tocar('parabens')`.
- Música: `tocarMusica()` ao entrar no jogo (ex.: `WelcomeScreen`), `pararMusica()` ao sair.

Passo 5 — Respeitar o botão "Som"

Quando o switch de Som mudar, chame `setSomLigado(valor)`.

PARTE 8 — Dublagem / narração (voz) — passo a passo

Objetivo: o jogo **fala** com a criança (instruções, letras, história).

Escolha do tipo de voz (do melhor pro mais simples)

1. **Voz humana gravada** (recomendado pra criança): grave no celular ou no **Audacity** (grátis). Soa mais acolhedor.
2. **TTS premium** (ElevenLabs, Google/Azure): vozes naturais em pt-BR; você digita e baixa o áudio.
3. **TTS no aparelho** (`expo-speech`): robótico, mas instantâneo — plano B.

Passo 1 — Roteiro de falas

Liste todas as frases: `clique_na_vogal` → "Clique na vogal!", `letra_a` → "A", `parabens` → "Parabéns!", `historia_1` → "Olá! Eu sou o Ziggy...", etc.

Passo 2 — Gravar/gerar e organizar

Salve em `assets/sounds/voz/` com o nome da chave: `clique_na_vogal.mp3`, `letra_a.mp3`, ...

Passo 3 — Serviço de voz (`src/services/voz.js`)

```
// src/services/voz.js
import { createAudioPlayer } from 'expo-audio';

let vozLigada = true;
export function setVozLigada(v) { vozLigada = v; }
```

```
const falas = {
  clique_na_vogal: createAudioPlayer(require('../assets/sounds/voz/clique_na_vogal.mp3')),
  letra_a: createAudioPlayer(require('../assets/sounds/voz/letra_a.mp3')),
  parabens: createAudioPlayer(require('../assets/sounds/voz/parabens.mp3')),
  // ...adicione todas as chaves...
};

export function falar(chave) {
  if (!vozLigada) return;
  const p = falas[chave];
  if (!p) return;
  try { p.seekTo(0); p.play(); } catch (e) {}
}
```

Passo 4 — Usar

- O botão  da fase chama `falar('clique_na_vogal')`.
- Na história (Parte 10), cada fala toca sua voz.
- O switch **Voz** chama `setVozLigada(valor)`.

Plano B (texto dinâmico)

```
npx expo install expo-speech
```

```
import * as Speech from 'expo-speech';
Speech.speak('Clique na vogal', { language: 'pt-BR', rate: 0.9 });
```

PARTE 9 — Animação e vida na tela — passo a passo

- **Animated** (já vem no React Native): loops de flutuar/balançar/pulsar. Suficiente pro MVP.
- **Lottie** (`lottie-react-native`): animações ricas de personagem (do After Effects ou de lottiefiles.com).

Lembre da Parte 6: pra animar, borboleta/árvore precisam ser **imagens separadas** (PNG

transparente).

A) Borboleta voando — `src/components/Borboleta.js`

```
import React, { useEffect, useRef } from 'react';
import { Animated, Easing } from 'react-native';

const BORBOLETA = require('../assets/images/borboleta.png');

export default function Borboleta({ style, tamanho = 34 }) {
  const x = useRef(new Animated.Value(0)).current;
  const y = useRef(new Animated.Value(0)).current;
  useEffect(() => {
    Animated.loop(Animated.sequence([
      Animated.timing(x, { toValue: 1, duration: 4000, easing: Easing.inOut(Easing.sin),
        useNativeDriver: true }),
      Animated.timing(x, { toValue: 0, duration: 4000, easing: Easing.inOut(Easing.sin),
        useNativeDriver: true }),
    ])).start();
    Animated.loop(Animated.sequence([
      Animated.timing(y, { toValue: 1, duration: 1200, useNativeDriver: true }),
      Animated.timing(y, { toValue: 0, duration: 1200, useNativeDriver: true }),
    ])).start();
  }, []);
  const translateX = x.interpolate({ inputRange: [0, 1], outputRange: [0, 120] });
  const translateY = y.interpolate({ inputRange: [0, 1], outputRange: [0, -22] });
  return (
    <Animated.Image source={BORBOLETA}
      style={[{ position: 'absolute', width: tamanho, height: tamanho }, style, { transform: [{
        translateX }, { translateY }] }] } />
  );
}
```

Use: `<Borboleta style={{ top: '30%', left: '40%' }} />`.

B) Balançar (árvore, arbusto) — `src/components/Balanca.js`

```
import React, { useEffect, useRef } from 'react';
import { Animated, Easing } from 'react-native';

export default function Balanca({ children, style }) {
  const r = useRef(new Animated.Value(0)).current;
  useEffect(() => {
    Animated.loop(Animated.sequence([
      Animated.timing(r, { toValue: 1, duration: 1800, easing: Easing.inOut(Easing.sin),
        useNativeDriver: true }),
      Animated.timing(r, { toValue: -1, duration: 1800, easing: Easing.inOut(Easing.sin),
        useNativeDriver: true }),
    ])).start();
  }, []);
  const rotate = r.interpolate({ inputRange: [-1, 1], outputRange: ['-3deg', '3deg'] });
  return <Animated.View style={[style, { transform: [{ rotate }] }]}>{children}</Animated.View>;
}
```

C) Personagem rico (Lottie)

```
npx expo install lottie-react-native
```

```
import LottieView from 'lottie-react-native';
<LottieView source={require('../assets/animacoes/ziggy_acena.json')} autoPlay loop style={{
  width: 200, height: 260 }} />
```

PARTE 10 — Roteiro / história interativa — passo a passo

Passo 1 — Roteiro (dados) — `src/data/historia.js`

```
const ZIGGY = require('../assets/images/ziggy.png');
```

```
export const introNatureza = [
  { personagem: ZIGGY, texto: 'Bem-vindo à Ilha da Natureza!', voz: 'historia_1' },
  { personagem: ZIGGY, texto: 'Aqui vamos aprender as vogais brincando. Vamos?', voz:
'historia_2' },
];
// ...crie introDeserto, introGelo, etc.
```

Passo 2 — Componente de diálogo — `src/components/Dialogo.js`

```
import React, { useState, useEffect } from 'react';
import { View, Text, Image, Pressable, StyleSheet } from 'react-native';
import { cores } from '../utils/cores';
import { fontes } from '../utils/tema';
import { falar } from '../services/voz'; // se ainda não tiver voz, comente esta linha

export default function Dialogo({ falas, onFim }) {
  const [i, setI] = useState(0);
  const fala = falas[i];
  useEffect(() => { if (fala && fala.voz) falar(fala.voz); }, [i]);
  function avancar() { if (i + 1 >= falas.length) onFim(); else setI(i + 1); }
  return (
    <Pressable style={styles.overlay} onPress={avancar}>
      {fala.personagem && <Image source={fala.personagem} style={styles.personagem}
resizeMode="contain" />}
      <View style={styles.balao}>
        <Text style={styles.texto}>{fala.texto}</Text>
        <Text style={styles.toque}>toque para continuar ▶</Text>
      </View>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  overlay: { ...StyleSheet.absoluteFillObject, backgroundColor: 'rgba(0,0,0,0.25)',
flexDirection: 'row', alignItems: 'flex-end', padding: 20 },
  personagem: { width: '28%', height: '70%' },
  balao: { flex: 1, backgroundColor: cores.laranja, borderRadius: 22, padding: 20, marginLeft:
8, marginBottom: 20, borderWidth: 3, borderColor: cores.branco },
```

```
texto: { fontFamily: fontes.texto, fontSize: 20, color: cores.branco, lineHeight: 28 },
toque: { fontFamily: fontes.subtitulo, fontSize: 13, color: cores.branco, opacity: 0.85,
marginTop: 10, textAlign: 'right' },
});
```

Passo 3 — Mostrar a história (ex.: na tela da ilha, uma vez)

```
import { useState } from 'react';
import Dialogo from '../components/Dialogo';
import { introNatureza } from '../data/historia';

const [mostrarHistoria, setMostrarHistoria] = useState(true);
// no return, por cima de tudo:
{mostrarHistoria && <Dialogo falas={introNatureza} onFim={() => setMostrarHistoria(false)} />}
```

PARTE 11 — Cronograma sugerido (Trello)

Colunas **A Fazer / Fazendo / Feito**, uma etiqueta de cor por pessoa. - **Agora:** cada uma faz as suas ilhas (Parte 3), **uma de cada vez**, cada uma numa branch. Modelo = Ilha da Natureza. - **Depois das ilhas:** exportar as artes que faltarem (Parte 6). - **Camada de polimento:** Parte 7 (som) → Parte 8 (voz) → Parte 9 (animação) → Parte 10 (história). - **Reta final:** ajustes e testes de ponta a ponta no celular.

Regra de sobrevivência: o MVP (cadastro/login, ilha, fases, estrelas, progresso salvo, painel dos pais) **já está pronto**. O resto (som, voz, animação, história) é o que transforma de "funciona" em "encanta". Faça na ordem acima; se faltar tempo, corte de trás pra frente.

APÊNDICE — Cola de comandos do Git

```
git status                                # o que mudei / em que branch estou
git checkout main && git pull origin main  # pega o mais novo
git checkout -b feat/ilha-deserto         # cria branch nova
git add . && git commit -m "mensagem clara"
```

```
git push origin feat/ilha-deserto
git merge main                # traz o mais novo da main pra sua branch
git log --oneline             # histórico resumido
```

Rodar o app (na pasta com `App.js`):

```
npm install                  # 1ª vez, ou quando alguém adicionou dependência
npx expo start -c           # abre o Expo (o -c limpa o cache); leia o QR no Expo Go
```

Vocês conseguem. O caminho continua o mesmo: rodar → ver funcionando → mudar uma coisinha → entender. Um passo de cada vez, sempre commitando o que já funciona. ❤️